

小组练习 2_SQL 聚合（GROUP BY 和 HAVING）

一、背景

XX 销售数据分析员主要负责销售分析工作。请使用 SQL 分析 sales 表，并通过 GROUP BY 和 HAVING 生成聚合指标。

表：sales

order_id	customer_id	category	amount	order_date
1	101	Electronics	500	2025-01-10
2	102	Apparel	100	2025-01-12
3	101	Electronics	300	2025-01-15
4	103	Apparel	150	2025-01-18
5	104	Electronics	800	2025-01-20

任务

- 准备数据库环境
- 编写一条 SQL 查询，按类别（category）sales 表进行分组，并返回类别名称（category name）、销售总额（total sales amount (sum of amount)），以及平均订单金额（average order amount (avg of amount)）。使用 GROUP BY 子句。
- 编写一条 SQL 查询，按 category 对 sales 表进行分组，但只返回销售总额（total sales amount (sum of amount)）超过 700 的类别。使用 GROUP BY 和 HAVING 子句。

二、准备数据库环境

1. 准备 sales 数据文件（sales.csv）

sales.csv 文件内容如下：

```
order_id,customer_id,category,amount,order_date
1,101,Electronics,500,2025-01-10
2,102,Apparel,100,2025-01-12
3,101,Electronics,300,2025-01-15
4,103,Apparel,150,2025-01-18
5,104,Electronics,800,2025-01-20
```

2. 准备数据库

- 本项目使用 sqlite3 数据库。

- 构建 Python 代码，使用 pandas 库从 sales.csv 中读取数据并写入 sales 数据库中。

Python 代码如下：

```
# 导入所需的库
import sqlite3
import pandas as pd
import os

# 获取当前脚本所在目录的绝对路径
current_dir = os.path.dirname(os.path.abspath(__file__))

# 1. 连接 SQLite 数据库（文件 sales.db 不存在会自动创建）
conn = sqlite3.connect(os.path.join(current_dir, "sales.db")) # 创建数据库连接
cursor = conn.cursor() # 创建游标对象

# 2. 读取 csv 文件
df = pd.read_csv(os.path.join(current_dir, "sales.csv")) # 读取 CSV 文件
print("CSV 原始数据预览：")
print(df) # 打印 CSV 数据预览

# 3. 将数据写入 SQLite 表 sales，如果存在先覆盖
df.to_sql(name="sales", con=conn, if_exists="replace", index=False)

# 4. 验证数据表是否创建成功
print("\n=== 数据库 sales 表全部数据 ===")
res = cursor.execute("SELECT * FROM sales;").fetchall() # 查询 sales 表的所有数据
# 遍历查询结果并打印每一行
for row in res:
    print(row)
```

代码执行结果如下：

```
CSV 原始数据预览：
  order_id  customer_id  category  amount  order_date
0         1          101  Electronics    500  2025-01-10
1         2          102    Apparel    100  2025-01-12
2         3          101  Electronics    300  2025-01-15
3         4          103    Apparel    150  2025-01-18
4         5          104  Electronics    800  2025-01-20

=== 数据库 sales 表全部数据 ===
(1, 101, 'Electronics', 500, '2025-01-10')
(2, 102, 'Apparel', 100, '2025-01-12')
(3, 101, 'Electronics', 300, '2025-01-15')
(4, 103, 'Apparel', 150, '2025-01-18')
(5, 104, 'Electronics', 800, '2025-01-20')
```

结论：通过观察得出，数据库中的数据与 sales.csv 文件中的数据一致。

三、任务 1：按类别分组，返回类别名称、销售总额、平均订单金额

1. 该任务对应的 SQL

```
SELECT
    category,
    SUM(amount) AS total_sales,
    AVG(amount) AS avg_order_amount
FROM sales
GROUP BY category;
```

2. 该任务对应的 Python 代码

```
# 5. 按分类分组，查询【**分类名、销售总额、平均订单金额**】
print("\n=== 按分类分组，查询【分类名、销售总额、平均订单金额】 ===")
# 执行 SQL 查询，按分类分组，计算销售总额和平均订单金额
res = cursor.execute("""
    SELECT
        category,
        SUM(amount) AS total_sales,
        AVG(amount) AS avg_order_amount
    FROM sales
    GROUP BY category;
""").fetchall()
# 遍历查询结果并打印每一行
for row in res:
    print(row)
```

3. 代码执行结果

```
=== 按分类分组，查询【分类名、销售总额、平均订单金额】 ===
('Apparel', 250, 125.0)
('Electronics', 1600, 533.3333333333334)
```

4. 分析及结论

分析数据，得出 sales 只有两个分类：Electronics 和 Apparel。分别计算这两个分组的数据，计算结果如下：

- Electronics 分组：
销售总额=500+300+800=1600，均值=1600/3≈533.33
- Apparel 分组：
销售总额=100+150=250，均值=250/2=125

观察运行代码后的结果，与以上计算值一致。

四、任务 2：按类别分组，仅返回销售总额超过 700 的类别

1. 该任务对应的 SQL

```
SELECT
    category,
    SUM(amount) AS total_sales,
    AVG(amount) AS avg_order_amount
FROM sales
GROUP BY category
HAVING total_sales > 700;
```

2. 该任务对应的 Python 代码

```
# 6. 分组后，只保留「销售总额大于 700」的分类
print("\n=== 分组后，只保留「销售总额大于 700」的分类 ===")
# 执行 SQL 查询，按分类分组，计算销售总额和平均订单金额，并筛选销售总额大于 700 的分类
res = cursor.execute("""
    SELECT
        category,
        SUM(amount) AS total_sales,
        AVG(amount) AS avg_order_amount
    FROM sales
    GROUP BY category
    HAVING total_sales > 700;
""").fetchall()
# 遍历查询结果并打印每一行
for row in res:
    print(row)
```

3. 代码执行结果

```
=== 分组后，只保留「销售总额大于 700」的分类 ===
('Electronics', 1600, 533.3333333333334)
```

4. 分析及结论

根据任务 1 的计算结果：

- Electronics 分组的销售总额=1600
- Apparel 分组的销售总额=250

得出满足销售总额超过 700 的类别只有 Electronics。
观察代码运行结果与以上结论一致。

五、SQL 优化

观察任务 1 和任务 2 执行结果，发现【平均订单金额】小数点后数字过多，需要让查询字段【avg_order_amount】固定保留 2 位小数。

这里没有采用 `ROUND(AVG(amount), 2)`： `ROUND` 返回的仍是浮点数（REAL），末尾的 0 不会被保留， `Apparel` 的均值 125.0 只会显示为 `125.0` 而非 `125.00`，无法做到「固定两位小数」。因此改用 `PRINTF('%0.2f', ...)` 按格式化字符串输出，两个分组的显示位数才能统一。需要注意的是， `PRINTF` 的返回值是文本类型，仅适用于展示；若后续还要参与数值计算，应保留原始的 `AVG(amount)`。

1. 任务 1 的 SQL 优化

```
SELECT
    category,
    SUM(amount) AS total_sales,
    PRINTF('%0.2f', AVG(amount)) AS avg_order_amount
FROM sales
GROUP BY category;
```

运行结果如下：

category	total_sales	avg_order_amount
Apparel	250	125.00
Electronics	1600	533.33

2. 任务 2 的 SQL 优化

```
SELECT
    category,
    SUM(amount) AS total_sales,
    PRINTF('%0.2f', AVG(amount)) AS avg_order_amount
FROM sales
GROUP BY category
HAVING total_sales > 700;
```

运行结果如下：

category	total_sales	avg_order_amount
Electronics	1600	533.33

六、小组成员角色与贡献

姓名	角色与贡献
王金波	完成代码、审查报告内容
丁玲	完成背景和准备数据库环境、搭建报告框架
刘海龙	完成任务 1：按类别分组，返回类别名称、销售总额、平均订单金额
李敏	完成任务 2：按类别分组，仅返回销售总额超过 700 的类别
何漪雯	完成 SQL 优化（任务 1 和任务 2）